

VETO: WP Spectra

Static and Dynamic Security Analysis of WordPress Plugins

Scan. Observe. Report.

Project Proposal

CS3002 Information Security

Project Advisor:
Sarah Abid Khan

Group Members:

Muhammad Ahmad Butt	23L-3059
Ali Ahmed	23L-3067
Muhammad Zain Tahir	23L-3078

National University of Computer and Emerging Sciences
Department of Computer Science
Lahore, Pakistan

Abstract

WordPress websites rely heavily on third party plugins to provide additional functionality. However, a malicious, compromised or vulnerable plugin can inject spam links, alter website content, create unauthorized accounts, or establish persistence. Identifying such threats can be difficult because some malicious activities only become visible when a plugin is executed.

This proposal introduces VETO: WP Spectra, a WordPress plugin security analysis system that combines static and dynamic analysis to investigate suspicious plugins. The system first examines a plugin package for suspicious code patterns, known signatures, obfuscation indicators and potentially risky functions. It then executes a copy of the plugin inside an isolated, disposable WordPress sandbox and observes its runtime behavior. Finally, it generates an evidence based report containing detected indicators, observed activities, affected files and a risk summary for the administrator.

VETO will use rule based detection and monitoring techniques rather than AI or large language models. Its primary goal is to provide administrators with a structured way to inspect suspicious plugins before trusting them on a production website.

1. Introduction

WordPress's plugin ecosystem enables rapid website development by allowing administrators to extend website functionality without building every feature from scratch. However, it also introduces security risks when a plugin is malicious, compromised, obfuscated or disguised as a legitimate tool.

Such plugins may insert SEO spam, modify website pages, create unauthorized administrator accounts, schedule persistent tasks or communicate with external hosts. Some of these activities may not be immediately visible through normal website usage, making suspicious plugins difficult to investigate.

The project is motivated by the author's practical experience developing WordPress websites for clients. In one case, a client's website had a suspicious spam SEO plugin named "dmzktif." The author recalls spending approximately three hours diagnosing the issue and cleaning the website. This experience highlighted the time and effort required to identify a suspicious plugin, understand its behavior and determine what changes it had made. This is reported as the author's experience, not as an independently verified malware attribution.

VETO addresses this problem through a combination of static and dynamic security analysis. Instead of relying only on examining plugin source code, it also executes a copy of the plugin in an isolated environment to observe its actual behavior. The resulting findings are combined into a single evidence based report.

2. Goals and Objectives

The objectives of VETO are organized into three components:

- **Scan:** Perform static analysis on submitted WordPress plugin ZIP files. Examine plugin files for known signatures, suspicious PHP patterns, obfuscation indicators, potentially risky functions and package anomalies. This analysis takes place before the plugin is executed.
- **Observe:** Perform dynamic analysis by installing and executing a copy of the plugin in an isolated, disposable WordPress environment. Observe its runtime behavior, including file modifications, database changes, account creation, scheduled tasks and outbound network connection attempts. This helps identify activities that may not be

apparent from static code inspection alone.

- **Report:** Generate a timestamped security report combining static analysis findings and observed runtime activities. The report will include supporting evidence, affected files, an event timeline and a risk summary that administrators can review and export.

3. Scope of the Project

The prototype will analyze WordPress plugins supplied by the user, either as ZIP files or as copies of installed plugins. Analysis will take place in a self hosted test environment where the owner has authorized security testing.

The project will focus on three main areas:

1. **Static plugin analysis:** Inspecting plugin packages for suspicious code patterns, known signatures, obfuscation indicators, risky PHP functions and package anomalies.
2. **Dynamic behavioral analysis:** Executing a plugin copy in an isolated WordPress sandbox and observing selected runtime activities.
3. **Evidence based reporting:** Combining static and dynamic findings into a structured report containing evidence, affected resources, observed events and a risk summary.

VETO will not claim to detect every malicious plugin. Signature based scanning may miss new or heavily obfuscated threats, and sandbox behavior may differ from behavior on a production website. Some legitimate plugins may also perform activities that appear suspicious, resulting in false positives. Findings will therefore include supporting evidence and will be presented for administrator review rather than treated as definitive proof of maliciousness.

The sandbox will execute a copy of the submitted plugin. It will not attempt to move an already running process from a production website into the sandbox.

The project explicitly excludes:

- Real time monitoring of production WordPress websites.
- Live HTTP traffic inspection or web application firewall functionality.
- Visitor routing, deception routing or decoy based traffic handling.
- Automatic malware removal, website cleaning or remediation.

4. Proposed System Workflow

The proposed workflow combines static analysis and dynamic behavioral analysis into one process.

I. Plugin Submission

An administrator submits a WordPress plugin ZIP file or a copy of an installed plugin for security analysis.

II. Static Analysis

VETO unpacks the plugin and examines its contents using YARA rules, SHA 256 hash checks and PHP pattern checks. It looks for known signatures, suspicious functions, obfuscation indicators and unusual package structures. The findings are recorded before execution.

III. Sandbox Initialization

VETO starts a fresh, disposable WordPress environment using Docker. It establishes a baseline of the environment's files and database before installing the submitted plugin. The sandbox operates with restricted privileges, resource limits and controlled network access.

IV. Controlled Plugin Execution

A copy of the plugin is installed and activated inside the sandbox. To observe more than activation time behavior, the system performs a predefined test sequence. This may include visiting selected frontend pages, performing selected administrative actions and triggering scheduled tasks where applicable. The plugin is observed for a predefined execution period.

V. Dynamic Behavioral Monitoring

During execution, VETO monitors selected runtime activities, including:

- File creation, modification and deletion.
- Database changes.
- Creation of unexpected user or administrator accounts.
- Changes to WordPress scheduled tasks (WP Cron).
- Outbound network connection attempts.

The system compares these observations against the initial baseline. Static indicators and runtime events are considered together rather than treating an individual event as definitive proof of malicious behavior.

VI. Security Reporting and Cleanup

VETO combines the static and dynamic findings into a timestamped report containing detected indicators, affected files, observed activities, an event timeline and a risk summary. The report is made available for administrator review and export. After the analysis is complete, the disposable sandbox is destroyed.

4.1 Sandbox Isolation and Safety Controls

To ensure that analysis of untrusted plugins is performed safely, the sandbox will follow these isolation rules:

- The sandbox will not connect to a production WordPress website or database.
- It will not contain real user credentials or sensitive production data.
- Outbound network access will be disabled by default or restricted to controlled test endpoints.
- The sandbox will run without privileged container permissions.
- Its temporary environment and data will be destroyed after analysis.

5. Evaluation Methodology

The prototype will be evaluated using a controlled set of test cases with clearly defined calculations, rather than an elaborate benchmarking framework.

5.1 Test Environment

- A separate WordPress installation will be used inside a Docker based sandbox.
- Testing will use benign plugins and controlled test plugins containing predefined suspicious behaviors.
- Simulated activities will include file modification, unauthorized test account creation, database changes, scheduled task creation and outbound connection attempts to a controlled test endpoint.

5.2 Evaluation Metrics

Metric	How it will be measured
Static detection precision	Correctly identified suspicious plugins / all plugins flagged as suspicious
Static detection recall	Correctly identified suspicious plugins / all known suspicious plugins in the test set
Dynamic event detection rate	Correctly recorded test behaviors / total test behaviors triggered
False positive rate	Benign test cases incorrectly flagged / total benign test cases
Report completeness	Observed events correctly included in reports / total events recorded

6. Initial Study and Work Done So Far

The project idea is grounded in the author's experience developing WordPress websites for clients. During one engagement, a suspicious plugin named "dmzktif" was associated with spam SEO behavior. Diagnosing the issue and cleaning the affected website took approximately three hours. This incident motivated the development of a tool that could help inspect suspicious plugins, understand their behavior and preserve investigation evidence.

An initial review of existing WordPress security solutions identified several related categories:

- Web application firewalls that protect websites against malicious requests.
- Malware and file scanners that inspect website files for suspicious content.
- Vulnerability scanners that identify known security issues in WordPress components.
- Remote website scanners that examine publicly accessible websites for security indicators.

These tools provide important security capabilities. VETO's proposed contribution is a focused workflow that combines static inspection of a plugin package with isolated execution, behavioral monitoring and evidence based reporting. This is a proposed workflow distinction, not a claim that existing products lack all static or dynamic analysis capabilities.

7. Comparative Analysis of Existing Tools

Wordfence Scan provides WordPress file scanning for malicious code, backdoors, suspicious URLs, file changes and vulnerable or outdated components [1].

Sucuri Security provides malware scanning, blocklist checks, security hardening features and firewall related protection [2], [3].

WPScan focuses on identifying known vulnerabilities in WordPress core, plugins and themes [4].

VETO is not intended to replace these products. Its proposed focus is a prototype workflow that accepts a plugin package, performs static analysis, executes a copy in an isolated WordPress sandbox, observes selected runtime behavior and produces a consolidated evidence report.

The comparison summarizes the main documented emphasis of the selected tools. Their capabilities may vary depending on configuration, subscription plan and deployment. Further investigation of tools offering isolated plugin execution and behavioral analysis will help refine the comparative evaluation during development.

8. Expected Outcomes

The expected outcomes of the project are:

- **Static Analysis Module:** A plugin scanner that examines uploaded packages and reports suspicious signatures, PHP patterns, obfuscation indicators and potentially risky functions.
- **Dynamic Analysis Sandbox:** A disposable WordPress environment that executes a copy of a submitted plugin and monitors selected runtime activities, following the isolation controls in Section 4.1.
- **Behavioral Monitoring Module:** A monitoring component that records file changes, database modifications, account creation, scheduled task changes and outbound network connection attempts.
- **Evidence Based Security Report:** An exportable report containing static findings, runtime observations, affected files, an event timeline and a risk summary.
- **Evaluation:** Testing with benign plugins and controlled test cases, as described in Section 5, to measure static detection precision and recall, dynamic event detection rate, false positive rate and report completeness.

References

[1] Wordfence, “Scan,” Wordfence Help Documentation. [Online]. Available: https://www.wordfence.com/help/scan/. [Accessed: 24 October 2026].

[2] Sucuri, “Sucuri Security: Auditing, Malware Scanner and Security Hardening,” WordPress.org Plugin Directory. [Online]. Available: https://wordpress.org/plugins/sucuri-scanner/. [Accessed: 24 October 2026].

[3] Sucuri, “Malware Scanning,” Sucuri Documentation. [Online]. Available: https://docs.sucuri.net/plugins/malware-scanning/. [Accessed: 24 October 2026].

[4] WPScan, “WordPress Security Scanner,” WordPress.org Plugin Directory. [Online]. Available: https://wordpress.org/plugins/wpscan/. [Accessed: 24 October 2026].